



M1 ISDS

Semestre 2

Chiffrage au CIR (Crédit Impôt et Recherche)
U.E : Stage de fin d'étude Master 1

Souleymane NDIAYE

Encadreurs : F. Mourlin, C. Costes

Août 2024

Table des matières

1	Introduction	4
2	Contexte et Problématique	4
2.1	Présentation du Crédit Impôt Recherche (CIR)	4
2.1.1	Historique et cadre juridique du CIR en France	4
2.1.2	Critères d'éligibilité et principales dépenses prises en compte	5
2.2	Description des enjeux et problématiques liés au chiffrage du CIR	5
2.2.1	Problématiques rencontrées par les entreprises dans la collecte, la classification et la présentation des données nécessaires pour justifier le CIR	5
2.2.2	Les difficultés spécifiques liées à l'extraction de données pertinentes à partir de documents divers (PDF, tableurs, textes riches, etc.)	5
2.2.3	L'importance d'une démarche de chiffrage rigoureuse pour éviter des redressements fiscaux	6
3	Méthodologie	6
3.1	Collecte des données	6
3.2	Architecture Logicielle	6
3.2.1	Module Lecture	7
3.2.2	Module Nettoyage	8
3.2.3	Module Importation	9
3.2.4	Module Main	10
3.3	Traitement et analyse des données	11
3.3.1	Utilisation de PySpark et MongoDB pour l'analyse des données	11
3.3.2	Module Calcul_Metrique	12
3.3.3	Validation et stockage des résultats	13
4	Visualisation des données	13
5	Résultats	15
5.1	Interprétation des résultats	15
5.2	Limites et recommandations	16
6	Conclusion	16
	Références	16
7	Annexes	17

Remerciements

Je tiens à exprimer ma profonde gratitude à mon tuteur de stage, M F. Mourlin, pour son encadrement, sa patience, et son soutien tout au long de ce projet. Son expertise et ses conseils avisés ont été d'une grande aide, et m'ont permis de progresser tant sur le plan technique que professionnel.

Je remercie également toute l'équipe du Ministère de l'enseignement supérieur et de la recherche plus particulièrement au département C1 pour l'accueil chaleureux et l'opportunité qui m'a été offerte de travailler sur un projet aussi stimulant. Ce stage m'a permis de découvrir des environnements technologiques innovants et de m'immerger pleinement dans le monde de l'entreprise.

Enfin, je souhaite exprimer ma reconnaissance à la Sorbonne Université ISUP pour la formation solide qu'elle m'a dispensée. Les connaissances acquises durant mon cursus ont été précieuses pour mener à bien ce projet, et ce stage a été l'occasion idéale pour les mettre en pratique dans un contexte professionnel.

Merci à tous ceux qui ont contribué de près ou de loin à la réussite de cette expérience enrichissante.

1 Introduction

Le Ministère de l'Enseignement Supérieur et de la Recherche (MESR) joue un rôle fondamental dans le développement et la promotion de la recherche scientifique en France. Sa mission principale est de superviser les politiques de recherche et d'innovation, de coordonner les établissements d'enseignement supérieur, et de soutenir les initiatives scientifiques et technologiques qui contribuent au progrès de la société. Le MESR est également responsable de la gestion des financements publics dédiés à la recherche, parmi lesquels le Crédit Impôt Recherche (CIR) occupe une place centrale. Le CIR est un dispositif fiscal majeur visant à encourager les entreprises à accroître leurs efforts de recherche et développement (R&D) en France. En tant que tel, il joue un rôle crucial dans le soutien à l'innovation et au développement technologique des entreprises françaises.

Ce stage, réalisé au sein du MESR, a pour objectif de développer un système d'information complémentaire pour améliorer la gestion et l'analyse des données liées aux opérations de recherche bénéficiant du CIR. Ce projet s'inscrit dans un contexte où l'extraction et la standardisation des informations provenant de divers documents (textes riches, tableurs, PDF) sont essentielles pour répondre à des questions spécifiques sur les dépenses et les activités de recherche et de développement des entreprises. La durée du stage est de trois mois, sous la supervision de l'équipe de la Direction Générale de la Recherche et de l'Innovation (DGRI) dirigée par Mme C. Giry, dans le service de l'Innovation du Transfert de Technologie et de l'Action Régionale (SITTAR) dirigé par Mme Estelle Dhont-Peltrault dans le département C1. Ce dernier gère des politiques d'incitation à la recherche et au développement dirigé par Mme C. Costes.

Les objectifs de ce stage sont les suivants :

- Collecter et traiter les données semi structurées relatives aux opérations de recherche déclarées par les entreprises dans un data lake.
- Développer un pipeline de traitement des données capable de nettoyer, convertir et enrichir ces données pour les importer dans un data lab NoSQL.
- Permettre la consultation et l'analyse de données structurées issues du data lab tel que répondre à des questions spécifiques posées par le MESR de manière assistée.
- Faire évoluer les pratiques du contrôle CIR afin que l'usage de ces données devienne systématique par la mise en place de ce pipeline Big Data.

2 Contexte et Problématique

2.1 Présentation du Crédit Impôt Recherche (CIR)

Le Crédit Impôt Recherche (CIR) est un mécanisme d'incitation fiscale destiné à encourager les entreprises à accroître leurs efforts de recherche scientifique et technique. Il permet aux entreprises de bénéficier d'un crédit d'impôt calculé sur la base des dépenses engagées pour des activités de Recherche et de Développement (R&D) éligibles. Ce dispositif est essentiel pour le financement des activités de recherche et d'innovation en France, en réduisant le coût de ces activités pour les entreprises.

2.1.1 Historique et cadre juridique du CIR en France

Le Crédit d'Impôt Recherche (CIR) a été institué par la loi de finances en 1983 en France, dans le but d'encourager les entreprises à investir dans la recherche et le développement (R&D). Ce

dispositif a évolué au fil des années, notamment avec l'introduction de la notion de crédit d'impôt en volume en 2008, et des ajustements concernant les taux et les plafonds applicables. Le cadre juridique du CIR est encadré par le Code général des impôts, notamment les articles 244 quater B et suivants, qui définissent les conditions d'éligibilité des dépenses de R&D.

Le CIR vise à stimuler l'investissement en R&D des entreprises en leur offrant un crédit d'impôt proportionnel aux dépenses éligibles. Cela permet de réduire le coût net de l'investissement en R&D et d'inciter les entreprises à développer des projets innovants, renforcer leurs capacités de recherche et améliorer leur compétitivité à l'échelle nationale et internationale.

2.1.2 Critères d'éligibilité et principales dépenses prises en compte

Les dépenses éligibles au CIR couvrent une large gamme de coûts associés à la R&D, notamment :

- les salaires des chercheurs et techniciens de recherche,
- les frais de sous-traitance pour les projets de R&D réalisés en collaboration avec des organismes de recherche agréés,
- les amortissements des équipements et installations de recherche,
- les dépenses de veille technologique, les frais de défense des brevets, et certains coûts de normalisation

Les activités doivent viser à lever des incertitudes scientifiques ou technologiques, et les entreprises doivent être en mesure de démontrer que les travaux réalisés répondent aux critères de la R&D tels que définis par le Frascati Manual de l'OCDE voir [3] .

2.2 Description des enjeux et problématiques liés au chiffrage du CIR

2.2.1 Problématiques rencontrées par les entreprises dans la collecte, la classification et la présentation des données nécessaires pour justifier le CIR

Les entreprises rencontrent souvent des difficultés pour collecter, classer, et présenter les données nécessaires à la justification du CIR. Ces informations sont fréquemment dispersées dans différents formats et documents (tableurs, PDF, fichiers texte), rendant le processus de collecte et de consolidation complexe. De plus, chaque type de dépense peut nécessiter un traitement spécifique pour s'assurer qu'elle est éligible au CIR.

Dans ce but, un modèle de dossier justificatif a été produit par le département C1 voir [2]. Mais ce document n'est pas toujours respecté par les porteurs de demande du CIR, car ils n'ont pas l'obligation de la respecter .

2.2.2 Les difficultés spécifiques liées à l'extraction de données pertinentes à partir de documents divers (PDF, tableurs, textes riches, etc.)

L'extraction des données pertinentes à partir de documents divers représente un défi majeur. Les fichiers sont souvent non structurés ou semi-structurés, ce qui complique leur traitement automatique. Par exemple, des informations critiques peuvent être dispersées dans des paragraphes de texte ou intégrées dans des formats de fichiers qui ne sont pas directement exploitables par des outils d'analyse de données traditionnels. Cela exige la mise en place de processus automatisés et robustes pour l'extraction, le nettoyage, et la structuration de ces données.

Dans le cadre de ce stage, les documents sont généralement au format PDF ou Excel, mais cette contrainte n'est pas suffisante pour assurer l'extraction.

2.2.3 L'importance d'une démarche de chiffrage rigoureuse pour éviter des redressements fiscaux

Une démarche rigoureuse de chiffrage est essentielle pour éviter des erreurs qui pourraient entraîner des redressements fiscaux. Les entreprises doivent s'assurer que leurs déclarations sont précises et conformes aux exigences légales, et que toutes les dépenses éligibles sont correctement identifiées et documentées. Une approche systématique et bien structurée du chiffrage permet de minimiser les risques de non-conformité et d'optimiser les avantages du CIR.

Des instructeurs du département C1 effectuent des contrôles de demandes de CIR. Ils effectuent une analyse des documents et produisent un rapport PDF ayant un format identifié, auquel des annexes Excel sont jointes. Ces documents seront les sources d'informations exploitées dans ce travail .

3 Méthodologie

3.1 Collecte des données

Les données nécessaires pour le chiffrage du CIR sont issues de fichiers Excel contenant plusieurs onglets correspondant aux différentes catégories de dépenses (amortissements, sous-traitance, brevets & cov, Normalisation, Personnels et jeunes Docteurs, II présent. synthétique, Somme à déduire). Ces fichiers ont été fournis par les entreprises dans le cadre de leur dossier justificatif du CIR validés par les instructeurs. Chaque onglet est structuré différemment, ce qui nécessite une adaptation spécifique pour l'extraction et la transformation des données.

3.2 Architecture Logicielle

Nous avons structuré notre approche sous la forme d'un pipeline de traitement, composé des modules suivants : Lecture, Nettoyage, Importation et Main . Ces modules ont été développés en Python pour gérer chaque étape du processus de manière distincte et efficace :

- **Lecture** : Ce module joue un rôle crucial dans le processus de gestion des données, en permettant une vérification préliminaire de leur qualité. En lisant et en inspectant les données des différentes feuilles Excel, il aide à identifier et corriger d'éventuelles anomalies avant de passer aux étapes de traitement ou d'analyse plus avancées.
- **Nettoyage** : Ce module est un outil pratique pour automatiser le processus de nettoyage des données issues de fichiers Excel. En supprimant les lignes inutiles, en gérant les valeurs manquantes, et en s'assurant que les colonnes sont correctement nommées, il facilite grandement la préparation des données pour une analyse ou une utilisation ultérieure.
- **Importation** : Ce module est un outil puissant pour l'automatisation de l'importation de données Excel dans MongoDB. Il combine la gestion des erreurs, et l'insertion dans la base de données, facilitant ainsi le flux de travail dans des environnements nécessitant une manipulation et une gestion de données structurées.

Ces modules sont intégrés dans un pipeline automatisé qui se déclenche pour insérer les données utiles dans la base, transformant ainsi des ensembles de données non structurées ou semi-structurées en les exportant vers le Data Lab, permettant ensuite d'effectuer des requêtes SQL.

Le Data Lab joue un rôle central dans ce processus en facilitant la transformation et la structuration des données. Il permet de préparer les données de manière efficace, garantissant ainsi leur qualité et leur conformité avec les standards nécessaires pour les analyses et les traitements futurs.

MongoDB a été choisi pour ce projet en raison de sa **flexibilité de schéma**, ce qui permet de gérer facilement des données structurées et semi-structurées sans contrainte rigide. Sa **scalabilité** permet de traiter efficacement de grands volumes de données. De plus, son **intégration native avec PySpark** facilite l'analyse distribuée des données directement depuis MongoDB. Enfin, MongoDB bénéficie d'une **large communauté** et de nombreux outils de support, rendant son utilisation pratique et bien supportée voir [1] .

La **FIG. 1** ci-dessous illustre la structure de notre pipeline Big Data. Le data lake initial correspond à un répertoire de fichier partagé sur le réseau du département C1. Nous avons choisi pour le data lab un serveur de données orientées document qui supporte des schémas de données dynamiques.

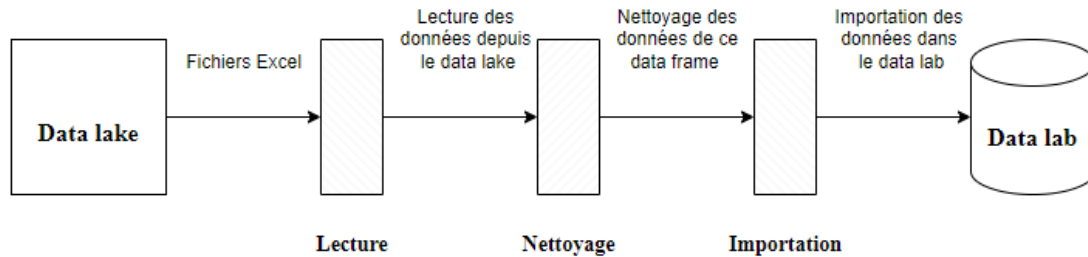


FIG. 1 – *Pipeline Big data*

3.2.1 Module Lecture

J'ai développé ce script Python en utilisant la bibliothèque **pandas** pour lire et analyser les données contenues dans les différentes feuilles d'un fichier Excel. Cette approche permet de vérifier l'intégrité des données avant leur traitement ou leur insertion dans une base de données.

Rôle du Module Le module a pour rôle principal de lire les différentes feuilles d'un fichier Excel et de convertir les données en **DataFrame** pour permettre une inspection approfondie. Cette vérification est essentielle pour détecter des anomalies telles que des colonnes mal nommées, des valeurs manquantes, ou des valeurs non numériques (**NaN**). Cela facilite ainsi la préparation des données pour les étapes ultérieures du projet.

Librairie Utilisée

- **pandas** : Cette librairie est utilisée pour lire les feuilles Excel et manipuler les données sous forme de **DataFrames**. Elle permet d'extraire facilement les données et de les analyser.

Processus Déroulé Le script suit un processus simple :

1. Lecture du Fichier Excel :

- Le fichier Excel est spécifié par son chemin, et la feuille à lire est passée en tant que paramètre à la fonction de lecture.
- La fonction `lire_excel` utilise **pandas** pour charger les données de la feuille spécifiée dans un **DataFrame**.

2. Vérification des Données :

- Une fois les données chargées, elles sont affichées à l'écran pour permettre une inspection manuelle.
- Cette étape permet de s'assurer que les colonnes sont correctement nommées, qu'il n'y a pas de valeurs manquantes significatives, et que toutes les valeurs sont correctement formatées.

Entrées du Module

- **file** : Le chemin d'accès complet au fichier Excel à lire.
- **feuille** : Le nom de la feuille à lire dans le fichier Excel.

Sorties du Module La sortie principale de ce module est un **DataFrame** contenant les données de la feuille Excel spécifiée. Ce **DataFrame** est ensuite utilisé pour afficher les premières lignes des données, ce qui permet de vérifier visuellement l'intégrité et la structure des données.

3.2.2 Module Nettoyage

J'ai développé ce script Python en utilisant la bibliothèque **pandas** pour manipuler et nettoyer des données provenant de fichiers Excel.

Rôle du Module Le rôle principal du module est d'automatiser la préparation des données en supprimant des lignes inutiles, en remplaçant les valeurs manquantes, et en renommant les colonnes des **DataFrames**. Cela permet une manipulation plus facile et plus cohérente des données dans les étapes suivantes de l'analyse.

Librairies Utilisées Le module utilise la bibliothèque **pandas**, qui est essentielle pour travailler avec des structures de données comme les **DataFrames** en Python. **Pandas** permet de lire, manipuler, et nettoyer efficacement des ensembles de données.

Classes et Méthodes

Classe **DataFrameCleaner**

Cette classe encapsule les méthodes nécessaires pour importer, nettoyer, et préparer les données.

- `__init__(self, file_path)` : Initialise la classe avec le chemin du fichier Excel à traiter.

- `supprimer_lignes(self, df, lignes)`: Supprime une ou plusieurs lignes d'un **DataFrame** en fonction des indices spécifiés et réindexe le **DataFrame**.
- `clean_dataframe(self, df)`: Nettoie le **DataFrame** en remplaçant les valeurs manquantes (**NaN**) par **None** et en s'assurant que les noms des colonnes sont des chaînes de caractères.
- `import_and_rename(self, sheet_name, first_line, id_name)`:
 - Importe un onglet spécifique d'un fichier Excel.
 - Remplit les valeurs manquantes par 0.
 - Renomme les colonnes en fonction d'une ligne spécifiée.
 - Nettoie le **DataFrame** et vérifie la présence d'une colonne spécifique (`id_name`).
 - Supprime les lignes où la colonne spécifiée a des valeurs manquantes, puis réindexe le **DataFrame**.
- `import_and_rename_pers_compt_en_hr(self, sheet_name='Personnels et jeunes Docteurs')`: Méthode spécifique pour importer et renommer les colonnes de l'onglet "Personnels et jeunes Docteurs", en supprimant les premières et dernières lignes non pertinentes.

Entrées du Module

- `file_path`: Le chemin vers le fichier Excel à traiter.
- `sheet_name`: Le nom de l'onglet à importer depuis le fichier Excel.
- `first_line`: L'indice de la ligne à utiliser pour les noms de colonnes.
- `id_name`: Le nom de la colonne qui doit être présente et nettoyée dans le **DataFrame**.
- `lignes`: Les indices des lignes à supprimer du **DataFrame**.

Sorties du Module Les méthodes renvoient des **DataFrames** nettoyés comme les Amortissements, la Sous-traitance, les Brevets & COV, la Normalisation, le II présent. synthétique et Personnels et jeunes Docteurs qui sont prêtes à être utilisées pour des analyses ultérieures ou des insertions dans une base de données.

3.2.3 Module Importation

J'ai développé ce script Python conçu pour importer et traiter des données provenant de fichiers Excel, puis les insérer dans une base de données MongoDB. Il combine les fonctionnalités de **pandas** pour la manipulation des données et de **pymongo** pour l'interaction avec MongoDB.

Rôle du Module Le module a pour rôle principal d'automatiser l'importation et l'insertion des données Excel dans MongoDB. Il permet de traiter plusieurs onglets d'un fichier Excel, d'appliquer des transformations de données spécifiques, et d'insérer ces données dans les collections appropriées de la base de données.

Librairies Utilisées

- **pandas**: Utilisée pour la manipulation et le nettoyage des données sous forme de **DataFrames**.
- **pymongo**: Utilisée pour établir une connexion avec MongoDB, gérer les collections, et insérer les données.

- **nettoyage**: Un module personnalisé (voir 3.1.4) contenant la classe **DataFrameCleaner**, qui est utilisée spécifiquement pour nettoyer et préparer le **DataFrame** Personnel et Jeunes Docteurs

Classes et Méthodes

Classe MongoDBImporter

Cette classe est responsable de l'importation des données dans MongoDB et de l'interaction avec le module de nettoyage des données.

- **__init__(self, mongo_uri, db_name, file_description, cleaner)**: Initialise la classe avec les informations de connexion à MongoDB, le nom de la base de données, une description des fichiers à traiter, et une instance de **DataFrameCleaner** pour le nettoyage des données.
- **insert_data(self, sheet_name, df)**: Insère un **DataFrame** dans la collection MongoDB correspondant au nom de l'onglet (**sheet_name**).
- **process_sheets(self)**: Traite tous les onglets décrits dans **file_description**, applique les transformations nécessaires sur les données, puis les insère dans MongoDB. Cette méthode gère également les exceptions potentielles liées aux clés manquantes ou aux erreurs d'insertion.
- **process_personnels_jeunes_docteurs(self)**: Traite spécifiquement l'onglet "Personnels et jeunes Docteurs", en appliquant un nettoyage additionnel avant l'insertion dans MongoDB.
- **close_connection(self)**: Ferme la connexion à la base de données MongoDB une fois toutes les opérations terminées.

Entrées du Module

- **mongo_uri**: L'URI de connexion à MongoDB.
- **db_name**: Le nom de la base de données dans MongoDB.
- **file_description**: Un dictionnaire décrivant les onglets du fichier Excel à traiter, avec les détails de nettoyage spécifiques (nom de l'onglet, première ligne des données, et nom de la colonne d'identifiant).
- **cleaner**: Une instance de la classe **DataFrameCleaner**, utilisée pour le nettoyage des **DataFrames**.

Sorties du Module Les données des **DataFrames** nettoyés comme les Amortissements, la Sous-traitance, les Brevets & COV, la Normalisation, le II présent. synthétique et Personnels et jeunes Docteurs sont insérées dans les collections correspondantes de MongoDB. Chaque méthode d'insertion renvoie un message de confirmation ou signale des erreurs potentielles.

3.2.4 Module Main

J'ai développé ce script Python qui combine les fonctionnalités de manipulation de données avec **pandas**, de nettoyage de données avec un module personnalisé, et d'importation dans une base de données MongoDB avec **pymongo**. L'objectif principal du script est de lire des données à partir d'un fichier Excel, de les nettoyer, puis de les insérer dans une base de données MongoDB.

Le script est conçu pour automatiser plusieurs étapes du traitement des données :

- Importer des données à partir de différents onglets d'un fichier Excel.
- Nettoyer les données en appliquant des transformations spécifiques, comme la suppression de lignes non pertinentes ou la gestion des valeurs manquantes.
- Insérer les données nettoyées dans des collections spécifiques de MongoDB pour un stockage à long terme.

3.3 Traitement et analyse des données

Cette section porte sur l'analyse des données qui ont été mises en œuvre dans ce stage pour fournir des observations utiles aux personnes du département C1.

3.3.1 Utilisation de PySpark et MongoDB pour l'analyse des données

Nous avons mentionné un premier intérêt du choix de notre data lab: le requêtage par SQL. Pour la suite de nos usages, il offre aussi la particularité de partitionner nos données pour distribuer les traitements d'analyse que nous faisons. Dans ce but, nous utilisons PySpark ,une plateforme de traitement de données distribuées, qui permet d'effectuer des requêtes complexes sur des volumes importants de données et de tirer des aperçues via des agrégations, calculs statistiques, et autres transformations. Grâce au connecteur Spark-MongoDB, il est possible de charger les collections MongoDB directement dans des DataFrames PySpark, facilitant ainsi l'analyse des données.

PySpark comprends 5 modules:

- **core** pour le traitement de fichier
- **sql** pour le traitement de table en mémoire
- **graphx** pour le traitement de graphe
- **streaming** pour l'échantillonnage de données au sein d'une boucle temporelle
- **mllib** pour la construction de modèle IA

Le module sql offre la possibilité d'interpréter des requêtes SQL sur notre data lab en utilisant le partitionnement des données. En effet les dataframe en mémoire sont construits comme des flux de lignes de données. Le terme consacré dans la librairie pyspark est Resilient Distributed Dataset (RDD) . En d'autre terme, cela signifie un flux de données utilisable en lecture seule par des traitements distribués voir [4] .

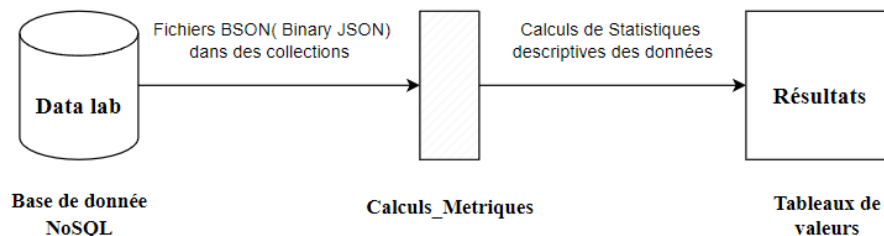


FIG. 2 – *Analyse descriptive*

3.3.2 Module Calcul_Metrique

Nous avons développé ce script Python en utilisant **PySpark** pour réaliser des analyses de données sur des informations stockées dans MongoDB. Il se concentre sur l'extraction, la transformation, et l'analyse de données en exploitant les fonctionnalités de **PySpark** pour le calcul distribué et la gestion des grands volumes de données.

Rôle du Script Le script a pour rôle principal de :

- Créer une session **Spark** pour interagir avec MongoDB.
- Charger des données depuis une collection MongoDB dans un **DataFrame** PySpark.
- Effectuer des analyses statistiques sur les données chargées, telles que le calcul de la moyenne, de l'écart-type et de la médiane.
- Analyser la distribution des équipements par type d'opération.

Librairies Utilisées

- **pyspark** : Utilisée pour créer des sessions Spark, manipuler les **DataFrames** PySpark, et effectuer des calculs distribués.
- **pymongo** : Bien que non explicitement utilisé ici, l'intégration de MongoDB est réalisée via les configurations de connexion dans Spark, en utilisant le connecteur MongoDB pour Spark.

Description des Étapes

- **Création de la Session Spark** : Le script commence par créer une session Spark en configurant les paramètres nécessaires pour se connecter à MongoDB. Ceci inclut la spécification des URI de connexion pour la lecture et l'écriture des données, ainsi que le package du connecteur MongoDB pour Spark.
Cette structure de données est distribuée sur tous les coeurs utilisés pour l'exécution. C'est dans cette session que les data frames sont déclarées afin d'être partagées.
- **Chargement des Données** : Les données sont ensuite chargées depuis MongoDB dans un **DataFrame** PySpark à l'aide du format `mongodb`. Ce **DataFrame** contient les informations sur les amortissements des équipements.
 - **Connexion à MongoDB** : Le script commence par créer une session Spark qui se connecte à une base de données MongoDB, spécifiquement aux collections contenant des informations sur les amortissements, les brevets & COV, et le personnels et jeunes docteurs.
 - **Chargement des Données** : Les données sont chargées dans des **DataFrames** PySpark à partir de MongoDB. Chaque **DataFrame** correspond à une collection distincte dans MongoDB.
 - **Analyse Statistique** : Des agrégations sont effectuées pour calculer des mesures telles que la moyenne, l'écart type, et la médiane des différentes colonnes d'intérêt. Des groupements par catégories sont également réalisés pour obtenir des répartitions de données.
 - **Calculs Spécifiques par Domaine** : Le script effectue des calculs spécifiques pour chaque domaine de données :
 - **Amortissements** : Calcul de la moyenne, de l'écart type et de la médiane des dotations annuelles, ainsi que la distribution des équipements par type d'opération et le total des amortissements affectés à la R&D.

- **Brevets et COV** : Analyse des dotations aux amortissements par prestataire, répartition des coûts de dépôt et de maintenance, et identification des brevets avec les coûts les plus élevés.
- **Personnel et Jeunes Docteurs** : Distribution des heures travaillées, analyse des coûts horaires et journaliers, et étude de l'influence des coefficients de la convention collective sur les salaires.
- **Affichage des Résultats** : Les résultats des calculs sont affichés sous forme de tableaux dans la console, facilitant ainsi l'interprétation et l'analyse des données.
- **Analyse de la Distribution par Type d'Opération** : Enfin, le script regroupe les équipements selon le type d'opération pour lequel ils ont été utilisés et calcule le nombre total d'équipements par type d'opération. Cette analyse est réalisée à l'aide de la fonction `groupBy` et de l'agrégation `count`.

Entrées du Script Les principales entrées du script incluent :

- L'URI de connexion à MongoDB pour la lecture et l'écriture des données.
- Les collections MongoDB contenant les données à analyser. Les connexions à ces collections sont configurées dans la session Spark au moyen de l'URI MongoDB approprié.

Sorties du Script Les sorties du script comprennent :

- Des tableaux de données affichés dans la console. Ces tableaux présentent les résultats des analyses statistiques et des agrégations réalisées sur les données des différentes collections MongoDB.
- Un tableau indiquant la distribution des équipements selon le type d'opération, avec le nombre total d'équipements pour chaque type.

Ce script est un outil puissant pour l'analyse de données volumineuses et complexes stockées dans MongoDB. En utilisant PySpark, il permet de manipuler et d'analyser efficacement les données en mémoire distribuée, tout en intégrant MongoDB comme source de données, ce qui en fait une solution robuste pour le traitement de données à grande échelle.

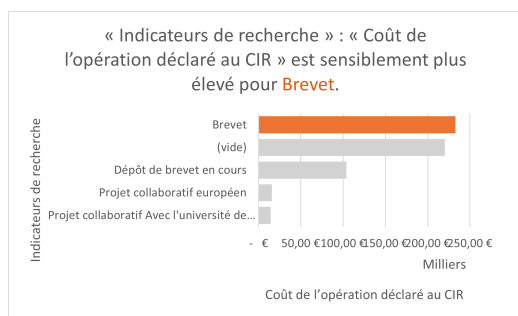
3.3.3 Validation et stockage des résultats

Les résultats des analyses sont validés par croisement avec les données sources et par vérification manuelle de la cohérence des agrégats calculés. Une fois validés, ces résultats sont stockés dans un Data Lake, où ils sont accessibles pour des consultations futures et pour une intégration éventuelle dans un système d'information élargi du MESR.

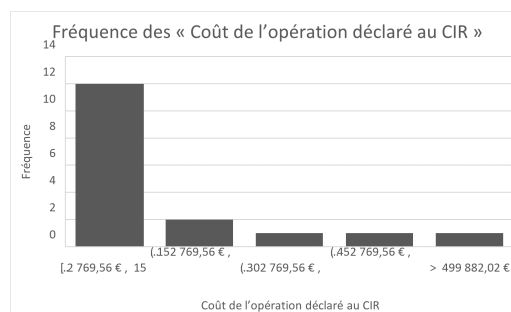
4 Visualisation des données

Pour explorer et représenter les données, j'ai utilisé Microsoft Excel et les bibliothèques Python, Matplotlib et Seaborn. Avec Excel, j'ai créé des graphiques simples pour obtenir rapidement une vue d'ensemble des distributions et des tendances. Ensuite, j'ai utilisé Matplotlib et Seaborn pour des visualisations plus détaillées, comme des graphiques en barres et des graphiques en violon, afin d'analyser la distribution des dépenses et d'autres statistiques.

Ces visualisations ont été intégrées dans le rapport pour illustrer et appuyer les conclusions de l'analyse.

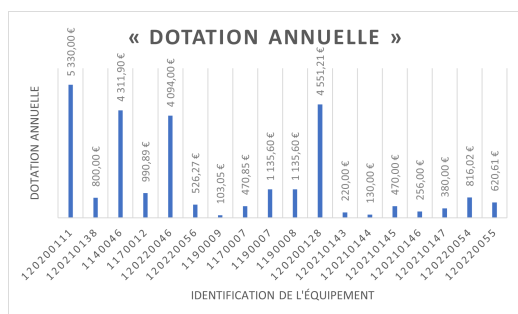


(a) Indicateurs de recherche “Coût de l’opération déclaré au CIR” est sensiblement plus élevé par Brevet

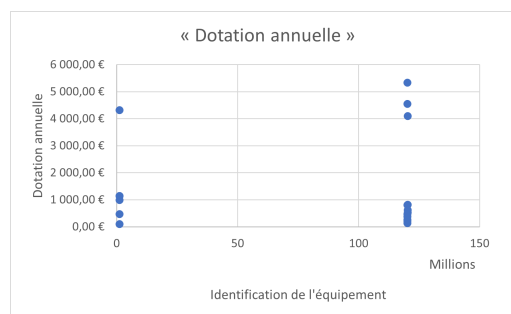


(b) Fréquence des “Coût de l’opération déclaré au CIR”

FIG. 3 – Coût l’opération déclaré au CIR

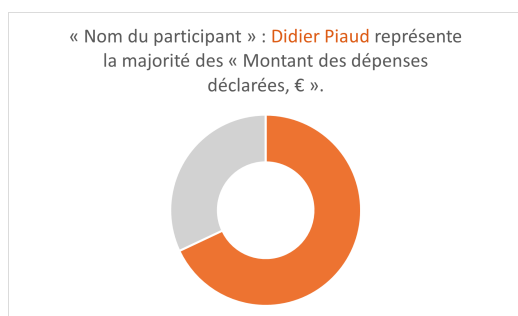


(a) Dotation annuelle

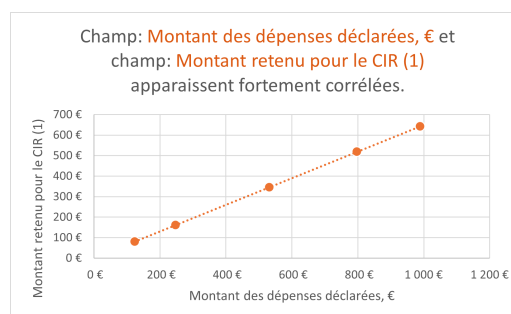


(b) Dotation annuelle

FIG. 4 – Identification de l’équipement



(a) Nom du participant qui représente la majorité



(b) Corrélation entre le Montant des dépenses déclarées, € et le Montant retenu pour le CIR (1)

FIG. 5 – Montant des dépenses déclarées, €

5 Résultats

5.1 Interprétation des résultats

Les résultats obtenus montrent des tendances significatives, telles que la concentration des dépenses sur certaines catégories spécifiques, et permettent de dégager des aperçus utiles pour l'optimisation du CIR.

A l'aide des modules **Nettoyage**, **Importation** et **Main** nous avons réalisé les résultats suivants

Résultats du Module de Nettoyage des Données Le module de nettoyage des données a permis de :

- Supprimer des lignes inutiles des DataFrames importés depuis les feuilles Excel, facilitant ainsi le traitement des données.
- Remplacer les valeurs manquantes (NaN) par **None**, garantissant ainsi une meilleure cohérence des données avant leur insertion dans MongoDB.
- Renommer les colonnes des DataFrames selon les spécifications, assurant une correspondance correcte des noms de colonnes pour l'analyse ultérieure.

Résultats du Module d'Importation dans MongoDB Le module d'importation a abouti aux résultats suivants :

- Les données nettoyées de plusieurs feuilles Excel comme les Amortissements, la Sous-traitance, les Brevets & COV, la Normalisation, le II présent. synthétique ont été insérées avec succès dans leurs collections MongoDB respectives.
- Les feuilles spécifiques telles que *Personnels et jeunes Docteurs* ont été traitées de manière distincte, avec une insertion correcte dans MongoDB après un nettoyage approprié.
- La structure des données dans MongoDB permet désormais une analyse efficace, grâce à la normalisation et à la préparation des données réalisées lors du nettoyage.

A l'aide du module **Calculs.Metriques** nous avons répondu certains questions

Analyse des Amortissements

- **Moyenne, Écart Type et Médiane :** Les statistiques sur les dotations annuelles d'amortissement montrent la répartition des coûts.
- **Distribution des Équipements :** Les équipements amortis sont répartis par type d'opération, identifiant les activités les plus consommatrices.
- **Amortissements en R&D :** Le total des amortissements affectés à la R&D est calculé, soulignant l'investissement en innovation.

Analyse des Brevets

- **Dotation Moyenne :** La dotation moyenne aux amortissements des brevets est déterminée.
- **Variation par Prestataire :** Les dotations varient selon les prestataires, et les plus fréquents sont identifiés.
- **Coûts de Brevet :** Les coûts de dépôt et de maintenance des brevets sont analysés, avec identification des brevets les plus coûteux.

Analyse du Personnel et Jeunes Docteurs

- **Distribution du Temps de Travail :** Le nombre d’heures/jours travaillés est analysé, avec des statistiques clés.
- **Coûts Horaires/Journaliers :** Répartition et analyse des coûts par fonction, en relation avec les coefficients/indices de la convention collective.
- **Comparaison des Salaires :** Différences de salaires et de coûts entre jeunes docteurs et autres personnels sont mises en évidence.

5.2 Limites et recommandations

Le projet a été marqué par plusieurs obstacles. Tout d’abord, l’installation des logiciels nécessaires, comme MongoDB, a pris près d’un mois et demi, retardant le début du travail. De plus, l’encadrement technique était limité, mon tuteur n’ayant pas assez de temps ni d’expertise en Python. En raison de restrictions de sécurité, je n’ai pu travailler que sur un nombre limité de documents transférés sur ma machine personnelle, et je n’ai pas pu traiter les fichiers PDF ou texte par manque de temps. Certaines limites ont été rencontrées, notamment liées à la qualité des données fournies par les entreprises. Des valeurs manquantes ou des formats hétérogènes ont parfois nécessité des ajustements manuels.

Pour pallier ces défis, il est recommandé de standardiser davantage les formats de soumission des données par les entreprises, préparer les outils en amont pour éviter les retards, renforcer l’encadrement technique, offrir un accès sécurisé à plus de documents pour une analyse complète..

6 Conclusion

Le travail réalisé dans le cadre de ce stage a permis de démontrer la faisabilité d’une extraction automatisée des données CIR, ainsi que l’intérêt de l’intégration de ces données dans un pipeline Big Data pour améliorer le contrôle et la gestion du CIR par le MESR.

Au-delà de ces réalisations techniques, ce stage a également été une opportunité précieuse d’apprentissage. J’ai pu me familiariser avec de nouvelles technologies telles que MongoDB et le langage de programmation PySpark, des outils essentiels dans le domaine du Big Data. L’accompagnement de mon tuteur a été déterminant, non seulement pour l’apprentissage de ces nouvelles compétences, mais aussi pour renforcer et mettre en pratique les connaissances acquises au cours de ma formation universitaire.

Il serait pertinent de développer des outils plus avancés pour l’extraction des données à partir de documents non structurés, tels que les PDF complexes. De plus, l’intégration de techniques d’apprentissage automatique pour prédire les risques de redressement ou pour classifier automatiquement les dépenses éligibles pourrait enrichir le système d’information développé.

Références

- [1] Anjali Chauhan. A review on various aspects of mongodb databases. *International Journal of Engineering Research & Technology (IJERT)*, 8(05):90–92, 2019.
- [2] Franck Debauge. *Guide pratique du crédit d’impôt recherche*. Editions Eyrolles, 2011.
- [3] Daniel Ker and Fernando Galindo-Rueda. *Frascati manual R&D and the system of national accounts*. OECD, 2017.

- [4] Raju Kumar Mishra and Sundar Rajan Raman. *PySpark SQL recipes*, volume 579. Springer, 2019.

7 Annexes

Les modules suivants sont annexés pour fournir des informations supplémentaires en lien avec le contenu principal de ce rapport.

Module Lecture

```
1 import pandas as pd
2 def lire_excel(file, feuille):
3     df = pd.read_excel(file, sheet_name=feuille)
4     return df
```

Module Nettoyage

```
1 import pandas as pd
2
3 class DataFrameCleaner:
4     def __init__(self, file_path):
5         self.file_path = file_path
6
7     def supprimer_lignes(self, df, lignes):
8         return df.drop(lignes, axis=0).reset_index(drop=True)
9
10    def clean_dataframe(self, df):
11        df = df.where(pd.notnull(df), None)
12        df.columns = [str(col) if not isinstance(col, str) else col for col
13                      in df.columns]
14        return df
15
16    def import_and_rename(self, sheet_name, first_line, id_name):
17        df = pd.read_excel(self.file_path, sheet_name=sheet_name)
18        df = df.fillna(0)
19        columns_name = list(df.iloc[first_line].values)
20        df.columns = columns_name
21
22        # Nettoyer le DataFrame avant insertion
23        df = self.clean_dataframe(df)
24
25        if id_name not in df.columns:
26            raise KeyError(f"'{id_name}' non trouvé dans les colonnes pour '{sheet_name}'")
27
28        df[id_name] = pd.to_numeric(df[id_name], errors="coerce")
29        df.dropna(subset=[id_name], inplace=True)
30        df = df.reset_index(drop=True)
31
32        return df
```

```

32
33     def import_and_rename_pers_compt_en_hr(self, sheet_name="Personnels_
jeunes_Docteurs"):
34         df = pd.read_excel(self.file_path, sheet_name=sheet_name)
35         columns_name = list(df.iloc[4, :].values)
36         df.columns = columns_name
37         df = self.supprimer_lignes(df, [0, 1, 2, 3, 4, 5, 70, 71, 72, 73,
74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85])
38         return df

```

Module Importation

```

1  import pandas as pd
2  from pymongo import MongoClient
3  from nettoyage import DataFrameCleaner
4
5  class MongoDBImporter:
6      def __init__(self, mongo_uri, db_name, file_description, cleaner):
7          self.client = MongoClient(mongo_uri)
8          self.db = self.client[db_name]
9          self.file_description = file_description
10         self.cleaner = cleaner # Instance de DataFrameCleaner
11
12     def insert_data(self, sheet_name, df):
13         collection = self.db[sheet_name]
14         collection.insert_many(df.to_dict('records'))
15         print(f"'{sheet_name}' a été inséré dans la base de données MongoDB
.")
16
17     def process_sheets(self):
18         for sheet_name, details in self.file_description.items():
19             if sheet_name == "file":
20                 continue
21             try:
22                 df = self.cleaner.import_and_rename(
23                     sheet_name=details["sheet_name"],
24                     first_line=details["first_line"],
25                     id_name=details["id_name"]
26                 )
27                 self.insert_data(sheet_name, df)
28             except KeyError as e:
29                 print(f"Erreur dans la clé {sheet_name}: {e}")
30             except Exception as ex:
31                 print(f"Erreur lors de l'insertion de '{sheet_name}': {ex}")
32
33     def process_personnels_jeunes_docteurs(self):
34         df = self.cleaner.import_and_rename_pers_compt_en_hr()
35         df = self.cleaner.clean_dataframe(df)
36         self.insert_data("Personnels_et_jeunes_Docteurs", df)
37

```

```

38     def close_connection(self):
39         self.client.close()

```

Module Main

```

1     # import pandas as pd
2     from pymongo import MongoClient
3     from nettoyage import DataFrameCleaner
4     from importation import MongoDBImporter
5
6     if __name__ == "__main__":
7         file_description = {
8             "file": r"C:\Users\HP\OneDrive\Bureau\Projet\Fichiers\Chauvin_\
9                 Arnoux_\CIR_2022_\Synthèse_financière.xlsx",
10            "Amortissements": {
11                "sheet_name": "Amortissements",
12                "first_line": 4,
13                "id_name": "Identification_de_l'équipement"
14            },
15            "Sous-traitance": {
16                "sheet_name": "Sous-traitance",
17                "first_line": 3,
18                "id_name": "N ° SIREN"
19            },
20            "Brevets_&_COV": {
21                "sheet_name": "Brevets_&_COV",
22                "first_line": 3,
23                "id_name": "Co ts_de_défense_déclarés,\n "
24            },
25            "Normalisation": {
26                "sheet_name": "Normalisation",
27                "first_line": 3,
28                "id_name": "Montant_des_dépenses_déclarées, "
29            },
30            "II_présent._synthétique": {
31                "sheet_name": "II_présent._synthétique",
32                "first_line": 0,
33                "id_name": "Co t_de_l'opération_déclaré_au_CIR"
34            }
35        }
36
37        # Initialisation de l'instance avec le chemin du fichier
38        cleaner = DataFrameCleaner(file_description["file"])
39
40        # Utilisation des méthodes pour importer et nettoyer des données spé
41        cifiques
42        for sheet_name, details in file_description.items():
43            if sheet_name == "file":
44                continue
45            df = cleaner.import_and_rename(details["sheet_name"], details["

```

```

45         first_line"], details["id_name"])
46     print(f"{sheet_name}_DataFrame:")
47     print(df.head())
48
49     # Pour l'onglet 'Personnels et jeunes Docteurs'
50     df_pers = cleaner.import_and_rename_pers_compt_en_hr()
51     print("Personnels_jeunes_Docteurs_DataFrame:")
52     print(df_pers.head())
53
54     # Initialisation de l'instance de DataFrameCleaner avec le chemin du
55     # fichier
56     cleaner = DataFrameCleaner(file_path=file_description["file"])
57
58     # Initialisation de l'importateur avec l'URI MongoDB, le nom de la base
59     # de données, la description du fichier, et l'instance de cleaner
60     importer = MongoDBImporter('mongodb://localhost:27017/', 'lundi',
61                               file_description, cleaner)
62
63     # Traitement des onglets et insertion dans MongoDB
64     importer.process_sheets()
65
66     # Traitement spécifique pour l'onglet 'Personnels et jeunes Docteurs'
67     importer.process_personnels_jeunes_docteurs()
68
69     # Fermeture de la connexion MongoDB
70     importer.close_connection()

```

Module Calculs_Metriques

```

1  # Pour la Collection Amortissements
2  from pyspark import SparkConf
3  from pyspark.sql import SparkSession
4  from pyspark.sql.functions import *
5  from pyspark.sql.types import *
6
7  # Créer une session Spark avec connexion      mongoDB
8  my_spark = SparkSession \
9      .builder \
10     .appName("myApp") \
11     .config("spark.mongodb.read.connection.uri", "mongodb://127.0.0.1/
12             vendredi.Amortissements") \
13     .config("spark.mongodb.write.connection.uri", "mongodb://127.0.0.1/
14             vendredi.Amortissements") \
15     .config('spark.jars.packages', 'org.mongodb.spark:mongo-spark-
16             connector_2.12:10.3.0') \
17     .getOrCreate()
18
19 # Charger les données dans un DataFrame PySpark
20 df_ammortissement = my_spark.read.format("mongodb").load()
21 from pyspark.sql import functions as F

```

```

19 print(df_ammortissement.columns)
20
21 # Calcul de la moyenne et de l'écart type
22 stats_df_ammortissement = df_ammortissement.agg(
23     avg(col("Dotation┐annuelle")).alias("averageCost"),
24     stddev_pop(col("Dotation┐annuelle")).alias("standardDeviationCost")
25 )
26
27 # Calcul de la médiane
28 median_df_ammortissement = df_ammortissement.approxQuantile("Dotation┐
    annuelle", [0.5], 0.05)
29
30 stats_df_ammortissement = stats_df_ammortissement.withColumn("medianCost",
    expr(str(median_df_ammortissement[0])))
31
32 stats_df_ammortissement.show()
33
34 # Distribution des équipements par type d'opération
35 df_ammortissement_operations = df_ammortissement.groupBy("Opération(s)┐pour
    ┐la(es)quelle(s)┐l'équipement┐a┐été┐utilisé").agg(
36     count("*").alias("total")
37 )
38
39 df_ammortissement_operations.show()
40
41 # Calculer la somme totale des montants des amortissements affectés la R
    &D
42 total_rd_amort_df = df_ammortissement.agg(
43     sum(col("Montant┐des┐amortissements┐affecté┐ ┐la┐R&D,\n┐pour┐l'
        OPERATION┐mentionnée\n ").alias("totalMontantsAmortissements")
44 )
45
46 total_rd_amort_df.show()
47
48 # Distribution des équipements par type d'opération
49 df_ammortissement_operations = df_ammortissement.groupBy("Opération(s)┐pour
    ┐la(es)quelle(s)┐l'équipement┐a┐été┐utilisé").agg(
50     count("*").alias("total")
51 )
52
53 df_ammortissement_operations.show()
54
55 # Pour la Collection Brevet & COV
56 from pyspark import SparkConf
57 from pyspark.sql import SparkSession
58 from pyspark.sql.functions import *
59 from pyspark.sql.types import *
60
61 # Créer une session Spark avec connexion MongoDB
62 my_spark = SparkSession \

```

```

63     .builder \
64     .appName("Brevet_COV_Analysis") \
65     .config("spark.mongodb.read.connection.uri", "mongodb://127.0.0.1/
        vendredi.Brevets_COV") \
66     .config("spark.mongodb.write.connection.uri", "mongodb://127.0.0.1/
        vendredi.Brevets_COV") \
67     .config('spark.jars.packages', 'org.mongodb.spark:mongo-spark-
        connector_2.12:10.3.0') \
68     .getOrCreate()
69
70 # Charger les données dans un DataFrame PySpark
71 df_brevet_cov = my_spark.read.format("mongodb").load()
72 from pyspark.sql import functions as F
73 print(df_brevet_cov.columns)
74
75 # 1. Dotation aux amortissements moyenne déclarée pour les brevets acquis (
    )
76 average_dotation_brevet = df_brevet_cov.agg(avg("Brevet_acquis:\ndotation_
    aux_amortissements_déclarée,\n ").alias("averageDotation"))
77 average_dotation_brevet.show()
78
79 # 2. Variation des dotations aux amortissements en fonction des
    prestataires
80 dotation_by_prestataire_brevet = df_brevet_cov.groupBy("Nom_du_prestataire\
    nutilisation_du_brevet_acquis") \
81     .agg(sum("Brevet_acquis:\ndotation_aux_amortissements_déclarée,\n ").
        alias("Total_Amortissement")) \
82     .orderBy(col("Total_Amortissement").desc())
83 dotation_by_prestataire_brevet.show()
84
85 # 3. Prestataires les plus fréquents pour l'utilisation des brevets acquis
86 frequent_prestataires_brevet = df_brevet_cov.groupBy("Nom_du_prestataire\
    nutilisation_du_brevet_acquis") \
87     .agg(count("*").alias("count")) \
88     .orderBy(col("count").desc())
89 frequent_prestataires_brevet.show()
90
91 # 4. Répartition des co ts de dép t et de maintenance par prestataire
92 costs_by_prestataire_brevet = df_brevet_cov.groupBy("Nom_du_prestataire\
    nutilisation_du_brevet_acquis") \
93     .agg(sum("Co ts_de_dép t_et_de_maintenance_déclarés,\n ").alias("
        totalCosts"))
94 costs_by_prestataire_brevet.show()
95
96 # 7. Brevet avec les co ts de dép t et de maintenance les plus élevés
97 max_cost_depot_brevet = df_brevet_cov.orderBy(col("Co ts_de_dép t_et_de_
    maintenance_déclarés,\n ").desc()).limit(1).select("N _de_dép t", "
        Co ts_de_dép t_et_de_maintenance_déclarés,\n ")
98 max_cost_depot_brevet.show()
99

```

```

100
101 # 8. Brevet avec les co ts de dépense les plus élevés
102 max_cost_depense_brevet = df_brevet_cov.orderBy(col("Co ts_de_dépense_dé
clarés,\n ").desc()).limit(1)
103 max_cost_depense_brevet.show()
104
105 # Pour la Collection Personnels et jeunes Docteurs
106 from pyspark import SparkConf
107 from pyspark.sql import SparkSession
108 from pyspark.sql.functions import *
109 from pyspark.sql.types import *
110
111 # Créer une session Spark avec connexion MongoDB
112 my_spark = SparkSession \
113     .builder \
114     .appName("Personnel_Jeunes_Docteurs_Analysis") \
115     .config("spark.mongodb.read.connection.uri", "mongodb://127.0.0.1/
vendredi.Personnels_et_jeunes_Docteurs") \
116     .config("spark.mongodb.write.connection.uri", "mongodb://127.0.0.1/
vendredi.Personnels_et_jeunes_Docteurs") \
117     .config('spark.jars.packages', 'org.mongodb.spark:mongo-spark-
connector_2.12:10.3.0') \
118     .getOrCreate()
119
120 # Charger les données dans un DataFrame PySpark
121 df_personnel = my_spark.read.format("mongodb").load()
122
123 from pyspark.sql import functions as F
124 print(df_personnel.columns)
125
126 # 1. Distribution du nombre d'heures ou de jours travaillés par an
127 distribution_hours_days = df_personnel.groupBy("Nombre_d'heures_ou_de_jours
travaillé(e)s_par_an(cf_DADS)") \
128     .agg(count("*").alias("count"))
129 distribution_hours_days.show()
130
131 # 2. Moyenne, médiane, écart type et variance du nombre d'heures ou de
jours travaillés par an
132 stats_hours_days = df_personnel.agg(
133     avg("Nombre_d'heures_ou_de_jours_travaillé(e)s_par_an(cf_DADS)").alias
("moyenne"),
134     stddev_pop("Nombre_d'heures_ou_de_jours_travaillé(e)s_par_an(cf_DADS)"
).alias("ecart_type"),
135     var_pop("Nombre_d'heures_ou_de_jours_travaillé(e)s_par_an(cf_DADS)").
alias("variance")
136 )
137 stats_hours_days.show()
138
139 # 3. Distribution du nombre total d'heures ou de jours par an hors opé
rations de R&D

```

```

140 distribution_hours_days_fonction = df_personnel.groupBy("Fonction") \
141     .agg(avg("Co t_horaire_ou\nCo t_journalier").alias("Co t_moyen"))
142 distribution_hours_days_fonction.show()
143
144 # 4. Distribution des co ts horaires ou journaliers
145 distribution_costs = df_personnel.groupBy("Co t_horaire_ou\nCo t_
    journalier") \
146     .agg(count("*").alias("count")) \
147     .orderBy(col("Co t_horaire_ou\nCo t_journalier"))
148 distribution_costs.show()
149
150 # 5. Répartition des différents postes/fonctions dans l'échantillon
151 distribution_fonction = df_personnel.groupBy("Fonction") \
152     .agg(count("*").alias("count"))
153 distribution_fonction.show()
154
155 # 6. Variation des co ts horaires ou journaliers en fonction des fonctions
156 costs_by_fonction = df_personnel.groupBy("Fonction") \
157     .agg(avg("Co t_horaire_ou\nCo t_journalier").alias("Co t_moyen")) \
158     .orderBy(col("Fonction"))
159 costs_by_fonction.show()
160
161 # 7. Influence des coefficients ou indices de la convention collective sur
    les salaires bruts annuels chargés
162 coefficient_influence_salaries = df_personnel.groupBy("coefficient_ou_
    indice_de_la_convention_collective_(2)\n\nFACULTATIF") \
163     .agg(
164         avg("Salaire_brut_annuel_chargé_(3)__(4)").alias("average_salary"),
165         count("*").alias("count")
166     ).orderBy("coefficient_ou_indice_de_la_convention_collective_(2)\n\
    nFACULTATIF")
167 coefficient_influence_salaries.show()
168
169 # 8. Relation entre coefficients/indices et co ts horaires ou journaliers
170 relation_coeff_costs = df_personnel.filter(col("coefficient_ou_indice_de_la
    _convention_collective_(2)\n\nFACULTATIF").isNotNull() & col("Co t_
    horaire_ou\nCo t_journalier").isNotNull()) \
171     .groupBy("coefficient_ou_indice_de_la_convention_collective_(2)\n\
    nFACULTATIF") \
172     .agg(avg("Co t_horaire_ou\nCo t_journalier").alias("averageCost")) \
173     .orderBy("coefficient_ou_indice_de_la_convention_collective_(2)\n\
    nFACULTATIF")
174 relation_coeff_costs.show()
175
176 # 9. Différence dans les salaires et les co ts horaires/journaliers entre
    les jeunes docteurs et les autres personnels
177 salary_cost_comparison = df_personnel.groupBy("Jeune_docteur_ou_autre_
    personnel") \
178     .agg(
179         avg("Salaire_brut_annuel_chargé_(3)__(4)").alias("averageSalary"),

```

```

180         avg("Co t_horaire ou\nCo t_journalier").alias("
           averageCostPerHourOrDay")
181     )
182 salary_cost_comparison.show()
183
184 # 10. Postes avec le nombre d'heures travaillées le plus élevé et le plus
      faible
185 highest_lowest_hours = df_personnel.groupBy("Fonction") \
186     .agg(sum("Nombre d'heures ou de jours travaillé(e)s par an (cf DADS)").
           alias("TotalHours")) \
187     .agg(max("TotalHours").alias("HighestHours"), min("TotalHours").alias("
           LowestHours"))
188 highest_lowest_hours.show()

```

Visualisation des données

```

1 # Pour le FIG. 4: Dotation annuelle
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from matplotlib import colors
6 import seaborn as sns
7
8 plt.figure(figsize=(10, 6))
9 sns.barplot(x=df_Ammortissement["Identification de l'équipement"], y=
      df_Ammortissement['Dotation annuelle en euros'], data=df_Ammortissement
      , palette='coolwarm')
10 plt.xlabel('Catégories')
11 plt.ylabel('Valeurs')
12 plt.title('Diagramme en barres avec Seaborn et palette coolwarm')
13 plt.show()

```